assignment3_part3

October 23, 2026

1 Assignment 3: Dealing with Vector and Matrix Real-World Data (Part III)

In Part II of the assignment, we have implemented a few similarity/distance functions. In part III, we will see how can we use them to analyze our restaurant rating data.

Let's first import the necessary packages and dependencies and load the dataset prepared in Part I of this assignment.

[1]:
```
import pandas as pd
import numpy as np

business_df = pd.read_csv('assets/montreal_business.csv')
business_df.set_index('business_id', inplace=True)

review_df = pd.read_csv('assets/montreal_user.csv')
rating_df = review_df.pivot_table(index=['business_id'], columns=['user_id'],␣
 ↪values='stars')
rating_df.fillna(0, inplace=True)

missing_business_id = set(business_df.index) - set(rating_df.index)
business_df.drop(list(missing_business_id), inplace=True)

business_df.head()
```

[1]:
```
                                        name  \
business_id
-1xuC540Nycht_iWFeJ-dw              Romados
-7bRnaHp7OHz8KW-THqP4w  Buffalo Bill Wings
-92cC6-X87HQ1DE1UHOx3w               Junior
-AgfhwHOYrsPKt-_xV_Ipg            Soupesoup
-FDkvLmwaBrtVgYFqEWeWA     Café Juicy Lotus

                                                                 categories  \
business_id
-1xuC540Nycht_iWFeJ-dw  Restaurants, Portuguese, Bakeries, Food, Chick…
-7bRnaHp7OHz8KW-THqP4w                            Restaurants, Chicken Wings
-92cC6-X87HQ1DE1UHOx3w     Pan Asian, Restaurants, Asian Fusion, Filipino
```

```
-AgfhwHOYrsPKt-_xV_Ipg                Sandwiches, Restaurants, Cafes, Soup
-FDkvLmwaBrtVgYFqEWeWA  Vegetarian, Food, Salad, Restaurants, Organic …

                        stars postal_code
business_id
-1xuC540Nycht_iWFeJ-dw    4.5     H2W 1C8
-7bRnaHp7OHz8KW-THqP4w    1.5     H4C 1G7
-92cC6-X87HQ1DE1UHOx3w    3.5     H3J 1M8
-AgfhwHOYrsPKt-_xV_Ipg    4.0     H2W 1G8
-FDkvLmwaBrtVgYFqEWeWA    3.5     H4A 1C9
```

[2]:
```
rating_df.head(5)
```

[2]:
```
user_id                --BumyUHiO_7YsHurb9Hkw  --NIc98RMssgy0mSZL3vpA  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -0zUpn_6kTWXHWzB85u7yA  -1MF2tosrw2WcCxeVNk81Q  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -1wbglcr6x1qrUbqP1YAIA  -3gey5u7j0qj5SZNrOHIQg  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -3s52C4zL_DHRK0ULG6qtg  -3uRUjmr1uEn-lLrhwRMCg  \
business_id
-1xuC540Nycht_iWFeJ-dw                     0.0                     0.0
-7bRnaHp7OHz8KW-THqP4w                     0.0                     0.0
-92cC6-X87HQ1DE1UHOx3w                     0.0                     0.0
-AgfhwHOYrsPKt-_xV_Ipg                     0.0                     0.0
-FDkvLmwaBrtVgYFqEWeWA                     0.0                     0.0

user_id                -4pYtFJbZYzKqmiOjGr4ag  -4tyD87vW73HGOmQlXk0uw  …  \
business_id                                                            …
```

```
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0  …
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0  …
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0  …
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0  …
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0  …

user_id                 zwOiJvD09N1dNf-ts7wscw  zwsVQwyxkz-s4rvBCaPyZw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0

user_id                 zyLIp7DRFqeIE1Grrh3Zng  zyYWUdaodH0h1jCZAvFRPg  \
business_id
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0

user_id                 zyaQNsI73cLOKttvGnj4bw  zyg4-MFtfPWmwucVazSjfw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0

user_id                 zzKsgIF472IyDpJxmHDIpw  zzgb9L3NBT9V8BGw_DcVYw  \
business_id
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0

user_id                 zznMmC74DNBcBG62gj3guA  zzo--VpSQh8PpsGVeMC1dQ
business_id
-1xuC540Nycht_iWFeJ-dw                    0.0                    0.0
-7bRnaHp7OHz8KW-THqP4w                    0.0                    0.0
-92cC6-X87HQ1DE1UHOx3w                    0.0                    0.0
-AgfhwHOYrsPKt-_xV_Ipg                    0.0                    0.0
-FDkvLmwaBrtVgYFqEWeWA                    0.0                    0.0

[5 rows x 11937 columns]
```

Suppose one of my favorite restaurants in Montréal is named *Modavie*, and I want to know how similar (in terms of customer ratings) it is compared to other restaurants. Let's see how we can do it.

[3]:
```
modavie_id = business_df[business_df.name.str.contains("Modavie")].index[0]
print(modavie_id)
```

```
QeFsIMcIFIJcOrpthYL8jg
```

This is a sanity check that restaurant "Modavie" does exist in our dataset.

1.0.1 Exercise 3. Find similar restaurants (15 pts)

Once we have a similarity metric implemented, we can use it to find the most similar vectors. In this case, can you find out which restaurants are most similar to Modavie based on dot product?

More specifically, can you implement the `find_max_dot_prod_restaurants` function to return the business_id of five restaurants that have the **largest** dot product with Modavie?

To do this, you need to compute the dot product between every restaurant's vector and modavie_vector. Then, store the dot products as a seperate column named `modavie_dot_prod` on the `business_df` dataframe. Then, you should output the five rows with the largest dot product (ranked in decreasing order and excluding Modavie itself).

HINT 1: You may refer to Assignment 2(Part III) Jaccard Similarity to think about how to calculate the similarity scores for all restaurants.

HINT 2: Use `np.dot` (instead of implementing your own dot product) to **greatly** speed up the execution.

[16]:
```
def find_max_dot_prod_restaurants(top_n):
    modavie_vector = rating_df.loc[modavie_id]
    business_df['modavie_dot_prod'] = rating_df.apply(
        lambda x:np.dot(modavie_vector, x)
        # YOUR CODE HERE
        #raise NotImplementedError()
        , axis=1)
    return business_df.sort_values('modavie_dot_prod', ascending=False).
↪drop(modavie_id).head(top_n)
```

We have provided the correct answers for you to verify your solution, if your code takes more than a few seconds to execute, please double check to ensure you are using `np.dot` instead of our DIY-ed `dot_prod`.

[17]:
```
max_sim_restaurants = find_max_dot_prod_restaurants(10)
max_sim_restaurants.name
```

[17]:
```
business_id
0W4lkclzZThpx3V65bVgig                  Schwartz's
5T6kFKFycym_GkhgOiysIw                 La Banquise
```

```
46Ld9Qc9nAx_AOjwclNZiw             Olive & Gourmando
s2I_Ni76bjJNK9yG60iD-Q        Maison Christian Faure
wl6w2VNMA4g6O9lu7kTvFA    Reuben's Deli & Steakhouse
wzugmCevnXuCMCF4upAf0w                          Kazu
cKdox2gt3L1Dbb7MpOPdWg            Au Pied de Cochon
9KmvrnyjWTr4sly0Dt770g                Jardin Nelson
J6qWt6XIUmIGFHX5rQJA-w                     L'Avenue
2wFSVah7gU9ImNuBHwIaDg                        Belon
Name: name, dtype: object
```

[18]:
```
answer = find_max_dot_prod_restaurants(5)
assert answer.iloc[0]['name'] == "Schwartz's"
assert answer.iloc[1]['name'] == "La Banquise"
assert answer.iloc[2]['name'] == "Olive & Gourmando"
assert answer.iloc[3]['name'] == "Maison Christian Faure"
assert answer.iloc[4]['name'] == "Reuben's Deli & Steakhouse"
```

1.0.2 Exercise 4. (10 pts)

We can also characterize the similarity on a larger scale. In fact, we are curious to know if Modavie's good ratings are more similar to other local area restaurants or to other French restaurants.

In this exercise, please calculate the mean **cosine similarity** between Modavie and other restaurants in the same local area (defined by the **first 3 digits** of the postal code), and other French restaurants (restaurants with "French" in its `categories`)

Hints: 1. Canada postal code uses a "AXB YCZ" pattern, where A, B, and C represent letters and X, Y, and Z represent numbers. For this exercise, we will only use the first 3 digits of a zip code, that is, "H2Y" for Modavie. 2. Again, you can see the wildness of real-world data: some food trucks are also listed as restaurants but they don't have fixed locations, let alone postal codes. For this exercise, we assume that they do not belong to any neighborhood. You can check for these and exclude them with the `.isna()` function. 3. Modavie itself should be excluded from the calculation. 4. We have prepared a NumPy-based `cosine_similarity` function for your convenience. 5. You can use the dataframe `.loc[index]` function to obtain the rating vectors from `rating_df`. These will accompany `modavie_vector` when calling the `cosine_similarity` function we've defined for you.

[25]:
```
business_df.loc[modavie_id].postal_code
```

[25]:
```
'H2Y 1Y6'
```

[28]:
```
def cosine_similarity(vec_x, vec_y):
    return np.dot(vec_x, vec_y)/(np.linalg.norm(vec_x) * np.linalg.norm(vec_y))

def similarity_with_local_restaurant():
    modavie_vector = rating_df.loc[modavie_id]
```

```
    local_restaurants_indices = business_df[
        # YOUR CODE HERE
        #raise NotImplementedError()
        business_df['postal_code'].str[:3].eq('H2Y')
        ].drop(modavie_id).index

    loc_cos_sims = []
    for index in local_restaurants_indices:
        # YOUR CODE HERE
        #raise NotImplementedError()
        if index != modavie_id:
            loc_cos_sims.append(cosine_similarity(modavie_vector, rating_df.
↪loc[index]))
    avg_cos_sim_local = sum(loc_cos_sims)/len(loc_cos_sims)


    french_restaurant_indices = business_df[
        # YOUR CODE HERE
        #raise NotImplementedError()
        business_df.categories.str.contains('French', na=False)
        ].drop(modavie_id).index

    french_cos_sims = []
    for index in french_restaurant_indices:
        # YOUR CODE HERE
        #raise NotImplementedError()
        if index != modavie_id:
            french_cos_sims.append(cosine_similarity(modavie_vector, rating_df.
↪loc[index]))
    avg_cos_sim_french = sum(french_cos_sims)/len(french_cos_sims)

    return avg_cos_sim_local, avg_cos_sim_french
```

[29]:
```
avg_cos_sim_local, avg_cos_sim_french = similarity_with_local_restaurant()
print(avg_cos_sim_local, avg_cos_sim_french)
```

```
0.019919428868434945 0.013618875603344313
```

[30]:
```
avg_cos_sim_local, avg_cos_sim_french = similarity_with_local_restaurant()
assert abs(avg_cos_sim_local - 0.019919428868434952) < 1e-8, "[Exercise 4]␣
↪Wrong value for avg_cos_sim_local."
```

[31]:
```
avg_cos_sim_local, avg_cos_sim_french = similarity_with_local_restaurant()
assert abs(avg_cos_sim_french - 0.01361887560334431) < 1e-8, "[Exercise 4]␣
↪Wrong value for avg_cos_sim_french."
```

After this exercise, you should be able to compute the similarity between many data vectors and

find the most similar vectors of a given vector. This can be used as the foundation of many advanced algorithms such as k-nearest neighbor classification, information retrieval, or clustering, which are the core of search engines or recommender systems. Computing the average similarity to selected groups of vectors (e.g., French restaurants) also provides a powerful tool to get deeper understanding of the data and to generate new features for downstream machine learning tasks. We encourage you to try these similarity metrics on your own data sets.